

Aether — Technical Architecture

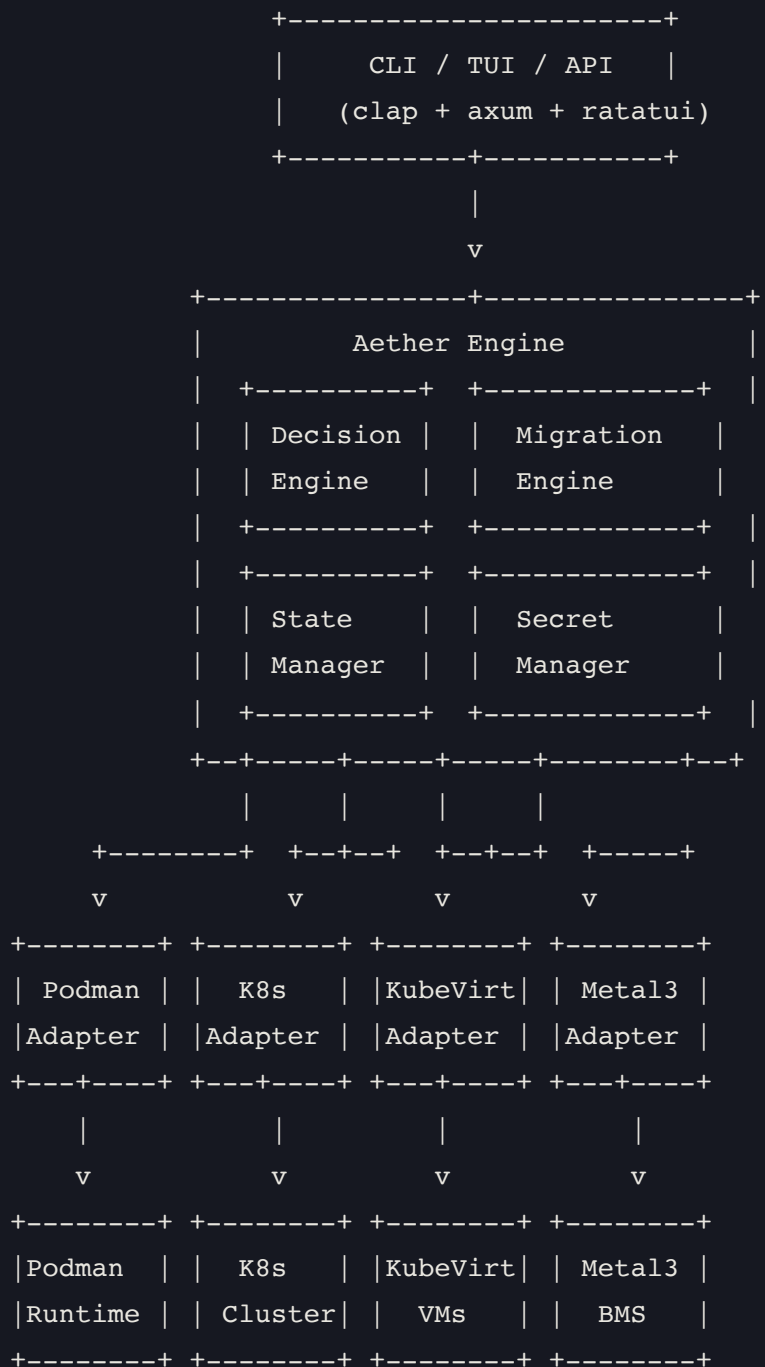
Deep Dive

Production HA topology: API replicas, Postgres state, Redis sessions, and runtime adapters — one portable control plane.

37K Lines of Rust • 916 Tests • 40+ Endpoints

Architecture Overview

Aether follows a layered adapter architecture. Every user action flows through the same pipeline: CLI or API, engine, adapter, runtime.



Each layer is independently testable. The adapter trait ensures new runtimes can be added without modifying the engine. The Decision Engine and Migration Engine operate on abstract workload models, making them runtime-agnostic by design.

Core Components

Four pillars form the foundation of the Aether engine, each responsible for a distinct concern.



Decision Engine

Evaluates workload specifications against runtime capabilities using a weighted scoring model. Considers resource limits, isolation requirements, scaling needs, and hardware affinity to select the optimal runtime. Supports manual overrides and constraint annotations.



Runtime Adapters

Each adapter implements a common trait with deploy, stop, status, logs, and migrate operations. Adapters translate Aether's abstract workload model into runtime-specific API calls. New runtimes are added by implementing the trait -- zero changes to the engine.



State Management

Workload state is persisted to a local JSON store with file-level locking for concurrency safety. State tracks deployment metadata, runtime binding, health status, and migration history. Designed for single-node reliability with optional remote sync.



Migration Engine

Orchestrates cross-runtime migrations with three strategies: immediate, blue-green, and rolling. Handles state transfer, health validation, traffic switching, and automatic rollback. Every migration is logged with full audit trail for compliance.



The adapter pattern is the key architectural decision. It makes Aether infinitely extensible without ever touching the core engine.

Runtime **Adapters**

Each adapter encapsulates the full lifecycle of a runtime, from deployment to teardown, behind a unified interface.

Adapter	Implementation	Timeout	Key Features
Podman	CLI subprocess via <code>tokio::process</code>	30s default	Pod creation, container lifecycle, health checks, volume mounts, network management, image pulls
Kubernetes	<code>kube-rs</code> client with kubeconfig discovery	60s default	Deployment/Service/ConfigMap CRUD, namespace isolation, resource limits, rollout status, log streaming
KubeVirt	<code>kube-rs</code> with CRD definitions	120s default	VirtualMachine lifecycle, cloud-init injection, disk management, live migration, console access
Metal3	<code>kube-rs</code> with BareMetalHost CRDs	300s default	Host provisioning, firmware config, RAID setup, network boot, hardware inventory, power management

Adapter Trait Interface

```
#[async_trait]
pub trait RuntimeAdapter: Send + Sync {
    async fn deploy(&self, spec: &WorkloadSpec) -> Result<DeployResult>;
    async fn stop(&self, name: &str) -> Result<()>;
    async fn status(&self, name: &str) -> Result<WorkloadStatus>;
    async fn logs(&self, name: &str, opts: &LogOptions) -> Result<String>;
    async fn migrate(&self, name: &str, target: RuntimeType) -> Result<Migr
```

```
fn runtime_type(&self) -> RuntimeType;
```

```
}
```

Security Architecture

Aether implements defense in depth with encryption, authentication, audit logging, and input validation at every layer.



AES-256-GCM Encryption

All secrets are encrypted at rest using AES-256-GCM with unique nonces. Key derivation uses Argon2id with configurable memory and iteration parameters.



API Authentication

The API server uses token-based authentication with configurable expiry. Tokens are validated on every request via middleware before reaching any handler.



Audit Integrity

Every operation is logged to an append-only audit trail with HMAC-SHA256 chain integrity. Tampering with any entry invalidates the entire chain.



File Permissions

State files, secret stores, and configuration are created with restrictive permissions (0600). Directory permissions are set to 0700 to prevent



Input Validation

All user input is validated and sanitized before processing. Workload names, image references, and resource limits are checked against strict allowlists and bounds.



K8s Resource Safety

Kubernetes deployments enforce resource limits, security contexts, and network policies by default. Privileged containers are blocked unless explicitly allowed.

unauthorized
access.

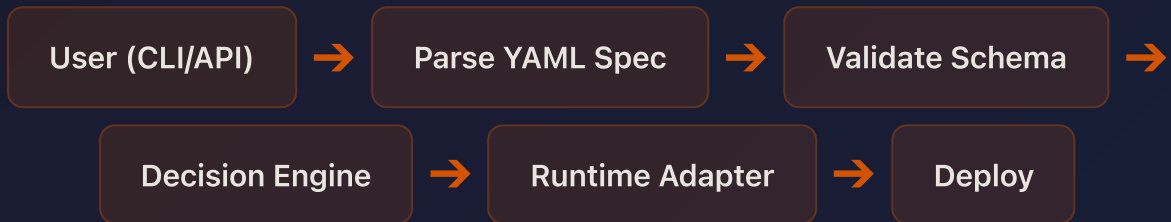


Security is not a feature -- it is a property of the architecture. Every layer assumes the layer above it may be compromised.

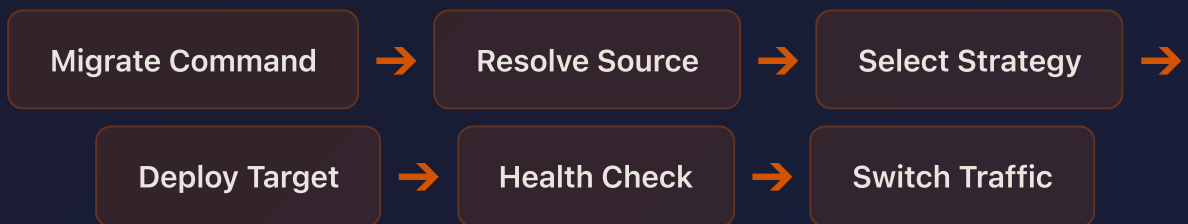
Data Flow

Understanding how requests move through Aether, from user intent to running workload and back.

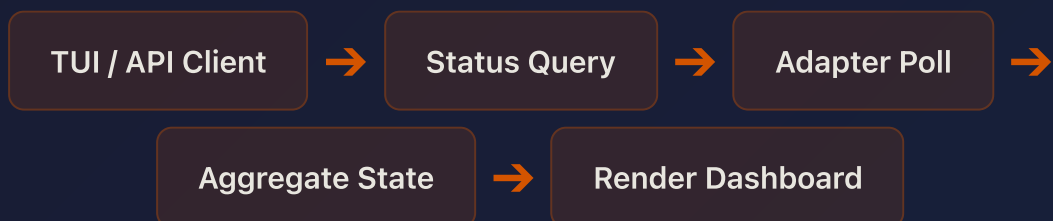
Deployment Flow



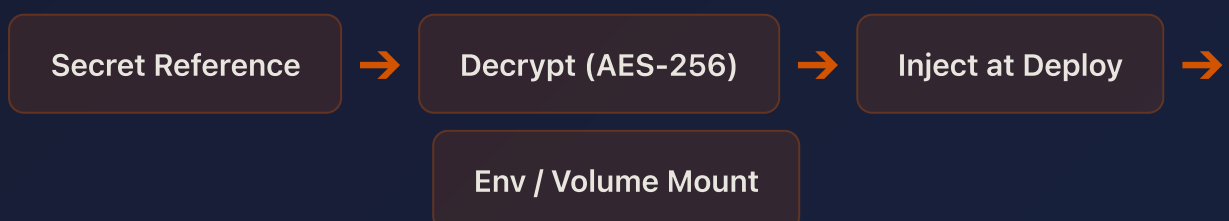
Migration Flow



Observation Flow



Secret Injection Flow



Every flow is fully async, powered by Tokio. Timeouts, retries, and circuit breakers protect against runtime failures at each stage.

API Server

Aether exposes a full-featured REST API built on Axum, providing programmatic access to every capability available through the CLI.

40+

REST Endpoints

Axum

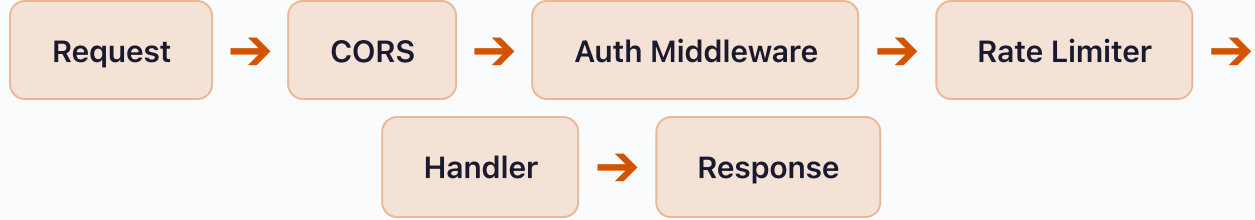
Web Framework

5090

Default Port

Endpoint Group	Methods	Description
/api/workloads	GET, POST, DELETE	List, deploy, and remove workloads across all runtimes
/api/workloads/:name/status	GET	Real-time workload status with health check results
/api/workloads/:name/logs	GET	Stream logs from any runtime with tail and follow options
/api/migrate	POST	Trigger cross-runtime migration with strategy selection
/api/secrets	GET, POST, DELETE	Manage encrypted secrets in the built-in secret store
/api/audit	GET	Query the tamper-evident audit log with filtering
/api/health	GET	Server health check for load balancer integration
/api/auth/login	POST	Authenticate and receive a session token

Middleware Stack



The API server shares the same engine as the CLI, ensuring feature parity. Every operation available through `aether` commands is also available via the REST API, enabling integration with CI/CD pipelines, custom dashboards, and third-party tooling.

Start Building

Extend Aether with custom adapters, integrate via the REST API, or deploy with the built-in toolchain.

Add a Custom Runtime Adapter

```
use aether::adapter::RuntimeAdapter;

pub struct MyCloudAdapter { /* ... */ }

#[async_trait]
impl RuntimeAdapter for MyCloudAdapter {
    async fn deploy(&self, spec: &WorkloadSpec) -> Result<DeployResult> {
        // Translate spec to your cloud's API
        // Deploy and return result
    }

    async fn status(&self, name: &str) -> Result<WorkloadStatus> {
        // Query your cloud for workload status
    }

    fn runtime_type(&self) -> RuntimeType {
        RuntimeType::Custom("my-cloud".into())
    }

    // ... implement remaining trait methods
}
```

Integrate via the REST API

```
# Deploy a workload via the API
curl -X POST http://localhost:5090/api/workloads \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name": "web-app", "image": "nginx:latest", "runtime": "auto"}'

# Check migration status
curl http://localhost:5090/api/workloads/web-app/status \
  -H "Authorization: Bearer $TOKEN"
```

Project Structure

```
aether/  
  src/  
    main.rs          # CLI entry point (clap)  
    engine.rs        # Core orchestration engine  
    decision.rs       # Runtime selection logic  
    migration.rs      # Cross-runtime migration  
  adapters/  
    podman.rs         # Podman adapter  
    kubernetes.rs     # Kubernetes adapter  
    kubevirt.rs       # KubeVirt adapter  
    metal3.rs         # Metal3 adapter  
  api/  
    server.rs         # Axum API server  
    routes.rs         # Endpoint definitions  
    middleware.rs     # Auth + rate limiting  
  security/  
    secrets.rs        # AES-256-GCM secret store  
    audit.rs          # HMAC-SHA256 audit chain
```

Build the Future of Infrastructure

Aether is open source, modular, and designed for extensibility. Whether you are deploying containers, VMs, or bare-metal servers, the architecture scales with your ambition.